

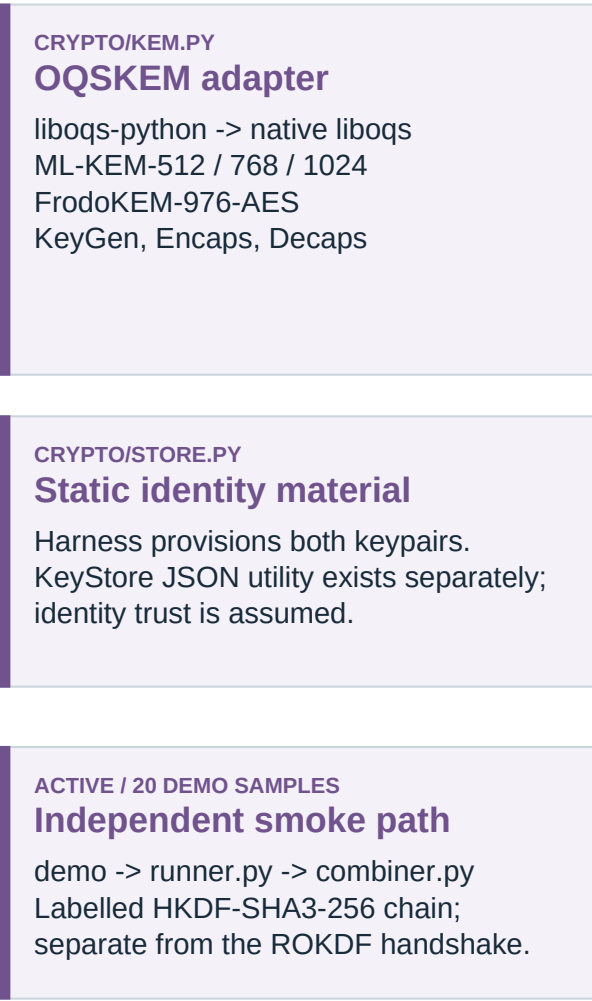
Hybrid QKD-PQC HAKE: system architecture

Implementation-grounded overview of execution paths, cryptographic dependencies, key delivery and research outputs.

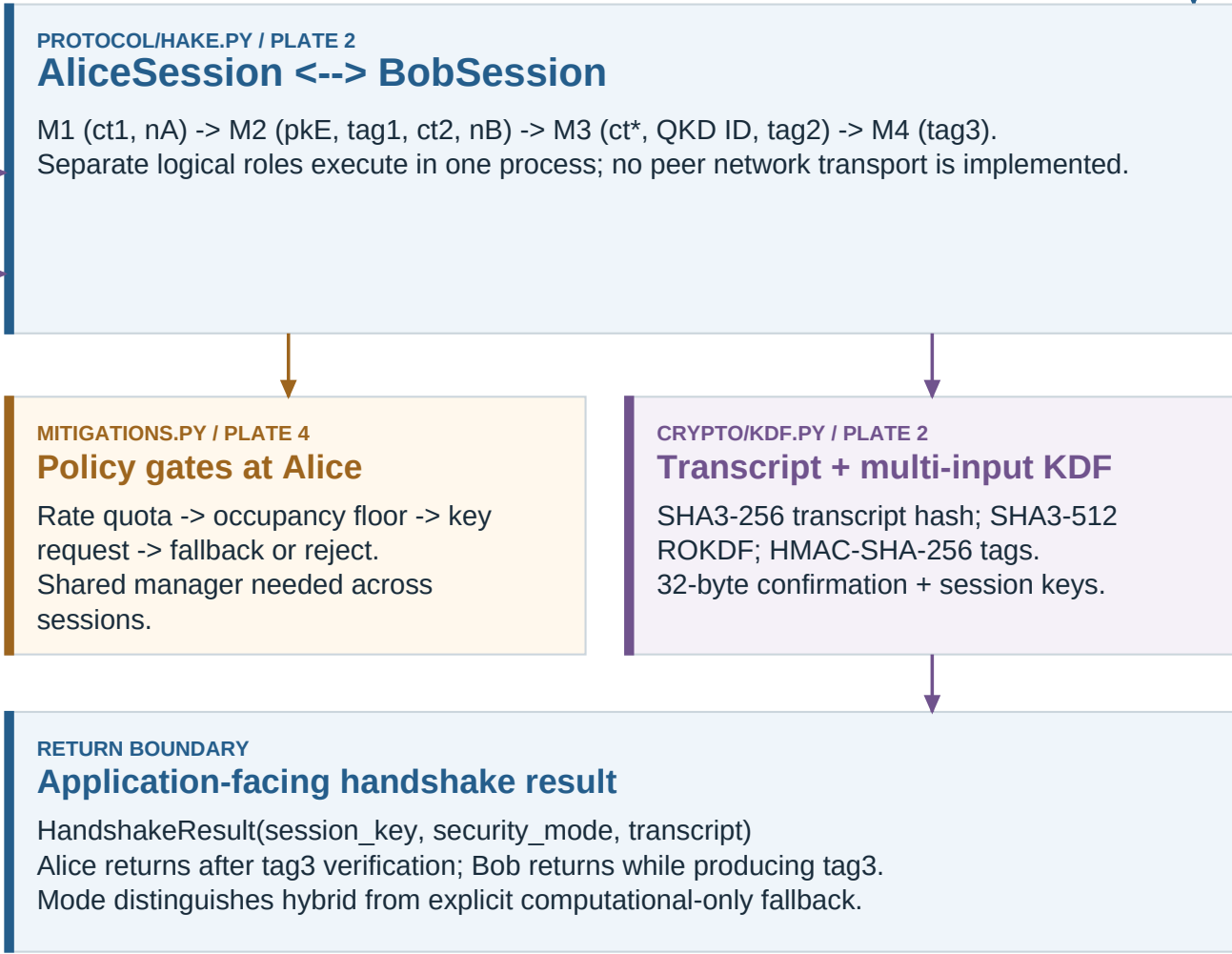
A / EXPERIMENT CONTROL AND ENTRY POINTS



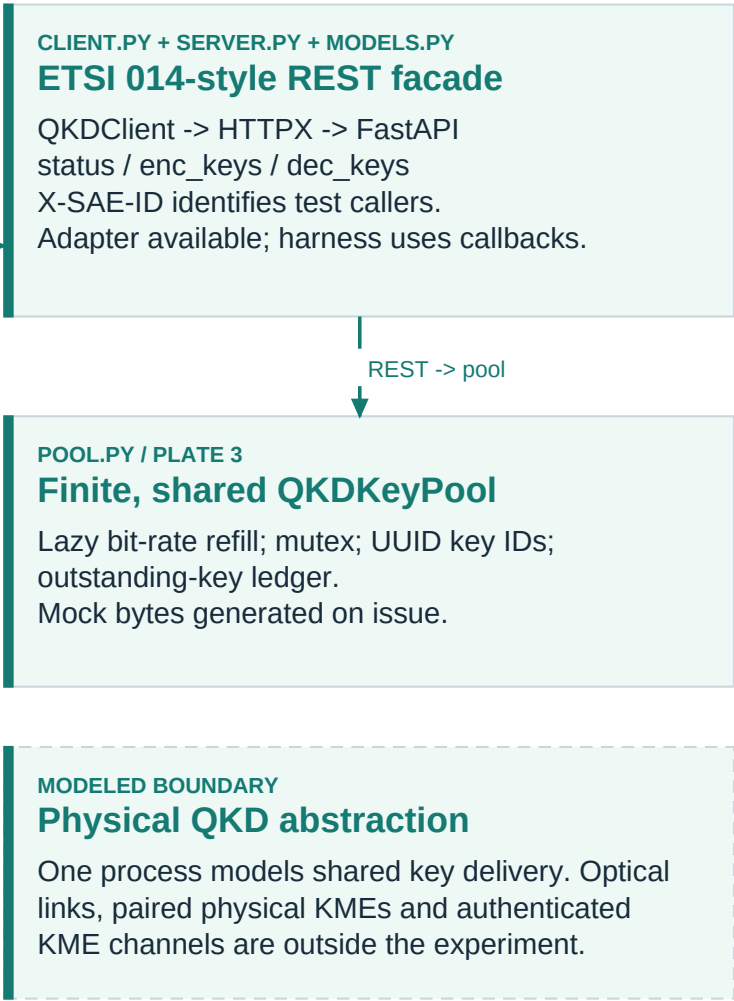
B / PRIMITIVES AND IDENTITY



C / LOCAL PROTOCOL EXECUTION



D / QKD SERVICE AND RESOURCE MODEL



E / MEASUREMENT AND SCIENTIFIC EVIDENCE

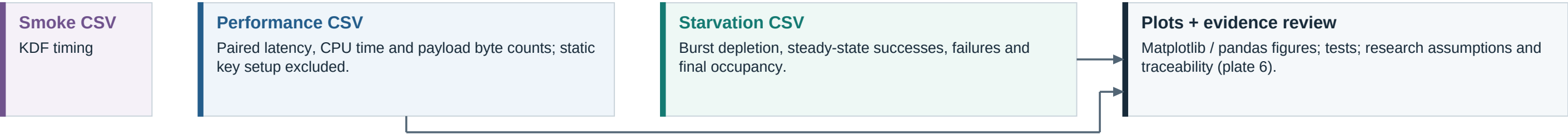
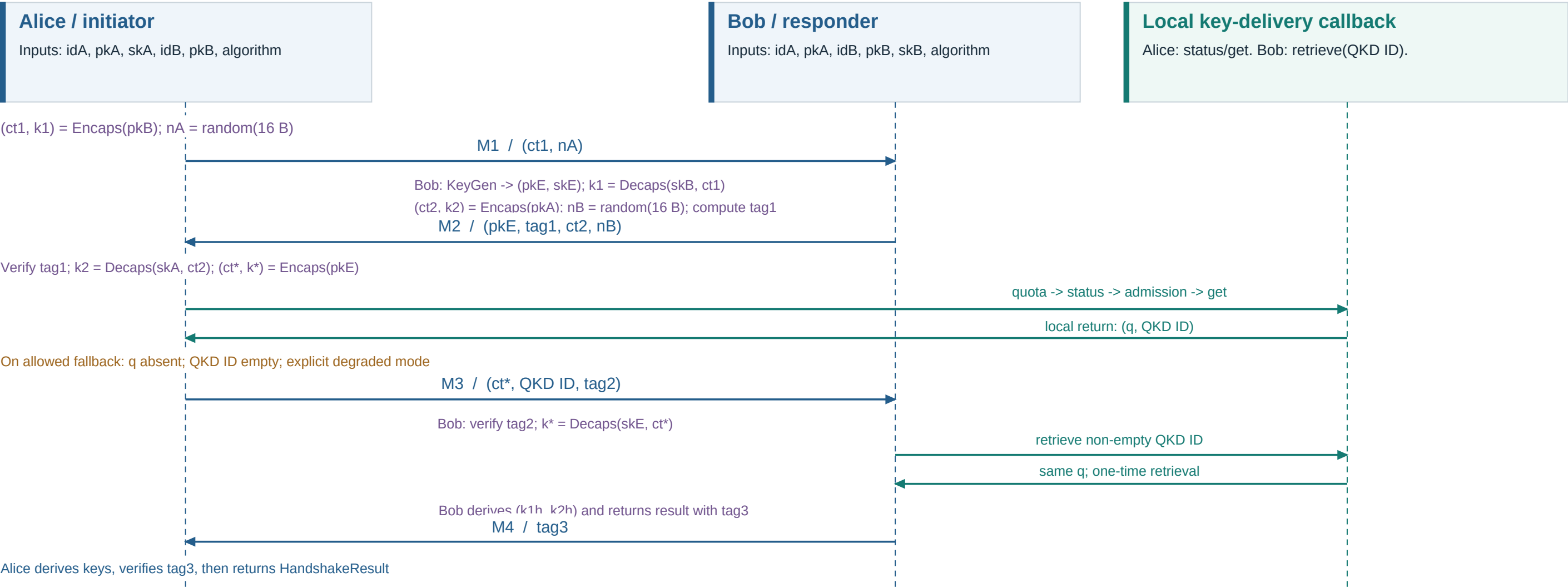


FIGURE 1. The three experiment paths share modules but measure different systems. Solid edges show active calls or data flow; dashed edges denote REST integration outside the default benchmark path, not measured network traffic.

Protocol execution and transcript-bound key schedule

Exact tuple ordering and derivation expressions from protocol/hake.py; this is the repository protocol, not a conformance claim.



A / CANONICAL CONTEXT AND DERIVATION

Length-prefixed transcript T

LP(version, idA, idB, algorithm, nA, nB, pkA, pkB, pkE, ct1, ct2, ct*, QKD ID, mode)
hT = SHA3-256(T); each LP field uses a 4-byte big-endian length.
Roles are ordered by the initiator/responder fields; mode is inferred from the QKD-ID path, then bound.

Confirmation tags

s = ct1 || ct2; MAC = HMAC-SHA-256
tag1 = MAC(k1, pkE || nB || s || 'A')
tag2 = MAC(k2, ID || ct* || s || 'B')
tag3 = MAC(k1h, LP(k1, ID, pkE, tag1, k2, tag2, hT))

ROKDF used by the handshake

cK = LP(idA, pkA, idB, pkB, pkE, k1, k2, k*, hT); cQ = LP(idA, idB, ID, hT)
kh = SHA3-512(label || LP(k*, cK, q_or_empty, cQ)); label = 'KEM-QKD-Hybrid KEX'
k1h = kh[0:32] (confirmation); k2h = kh[32:64] (session key)

Observable acceptance semantics

No final Alice-to-Bob acknowledgement. Returned transcript = T || tag3; tag1/tag2 are covered by confirmation but not returned in T. Empty QKD input is valid only on an explicit fallback path.

FIGURE 2. KEM public keys and QKD delivery are provisioned assumptions. Secret values stay in local process state; only the QKD identifier crosses the peer message boundary. This plate transcribes implementation behavior.

QKD key delivery, finite supply and pool accounting

One thread-safe in-memory pool models shared KME state; available-key credit and issued-key storage have different lifetimes.

A / REST CONTRACT (BASE PATH /api/v1/keys)

QKDCIENT.GET_STATUS()
Get status

GET /{slave}/status
Returns available count, depth, key size, SAE IDs and mock KME IDs.
Refill is evaluated on demand.
GET /health is a separate liveness endpoint.

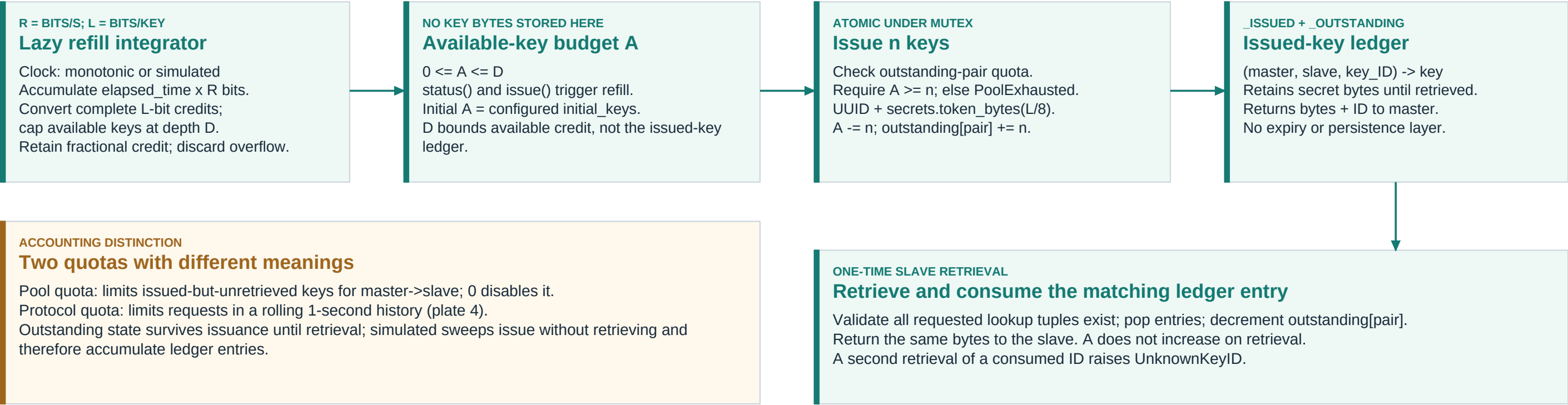
QKDCIENT.GET_KEY()
Get key

POST /{slave}/enc_keys: {number, size}
GET variant issues one key.
Returns [{key_ID, base64(key)}].
POST schema: 1..128 keys; configured size must match.

QKDCIENT.RETRIEVE_KEY()
Get key with IDs

POST /{master}/dec_keys: {key_IDs}
GET variant retrieves one key_ID.
Bob presents the ID obtained from Alice; lookup also binds the ordered SAE pair.

B / RESOURCE STATE AND LIFECYCLE



C / SERVICE BOUNDARY, ERRORS AND EXPERIMENT ASSUMPTIONS

TRUST BOUNDARY
Wire representation and identity

Caller supplies X-SAE-ID. Keys are base64-encoded in JSON, not encrypted by encoding. Local server binds 127.0.0.1:8000.
Authenticated HTTPS / SAE authorization and separate KME replication are outside the current service.

FAILURE SURFACE
Error mapping

401: missing X-SAE-ID
400: unsupported key size / unknown ID
503: pool exhaustion / pool quota
422: request schema validation
HTTP client raises on unsuccessful status.

MODEL
Finite-pool interpretation

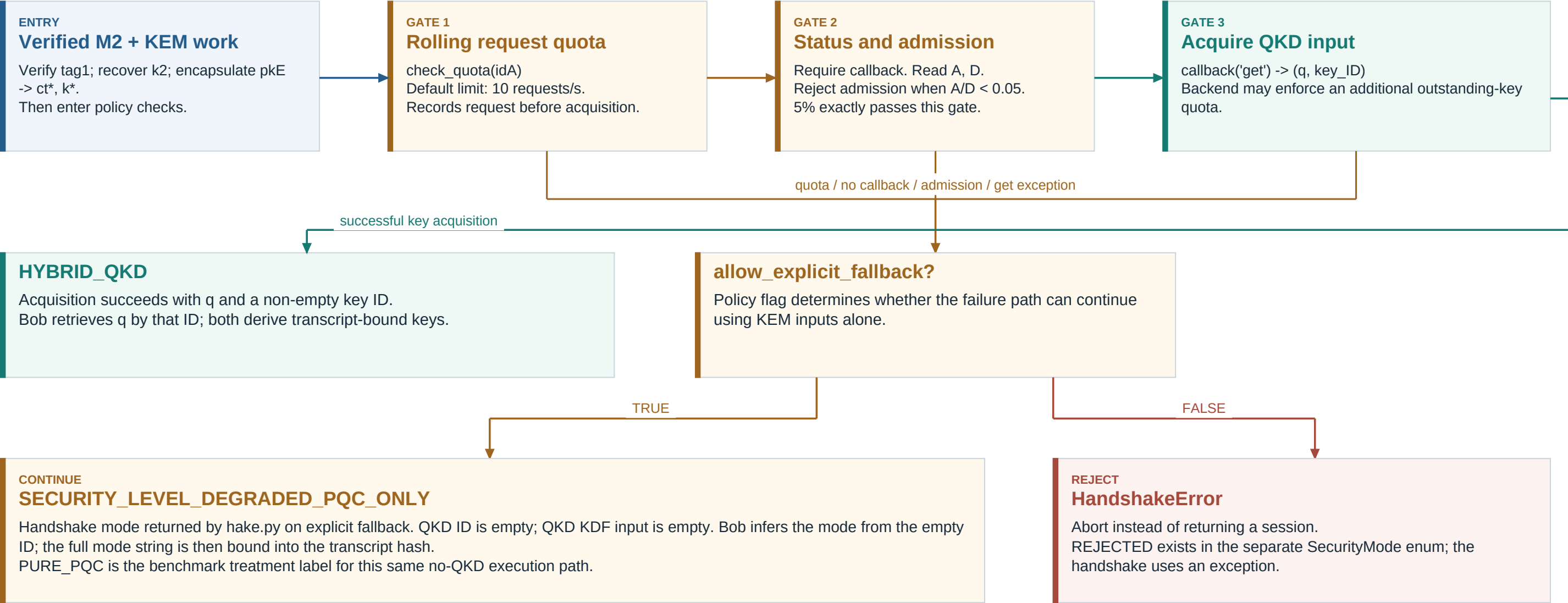
Default: R = 10,000 bit/s; L = 256 bits;
D = A0 = 128 keys.
Supply ceiling: R/L = 39.0625 keys/s.
This is a software supply model, not a quantum link or measured optical entropy source.

FIGURE 3. The pool is a rate-limited key-credit model with random bytes created at issue time. The available count is bounded; the outstanding ledger is not bounded when its quota is disabled. All pool mutations use one mutex.

Starvation policy, admission and security-mode outcomes

Decision order in AliceSession.message3; policies control access to QKD after the initial KEM work has already executed.

A / CONTROL FLOW



B / CONTROL SCOPE AND RESEARCH INTERPRETATION

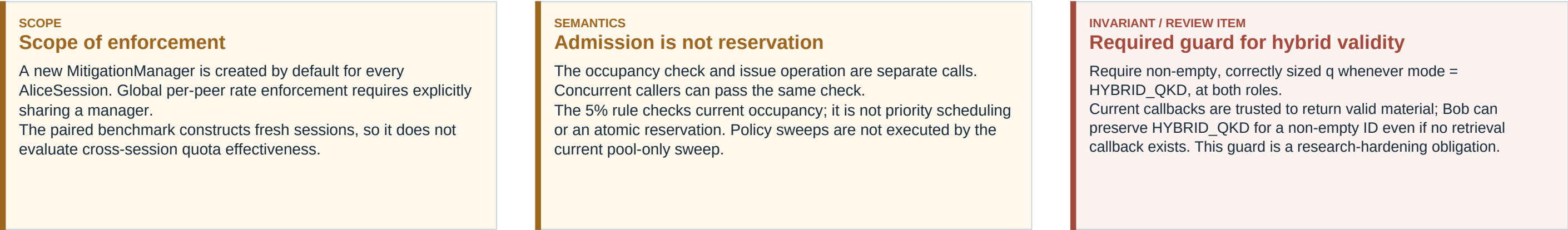
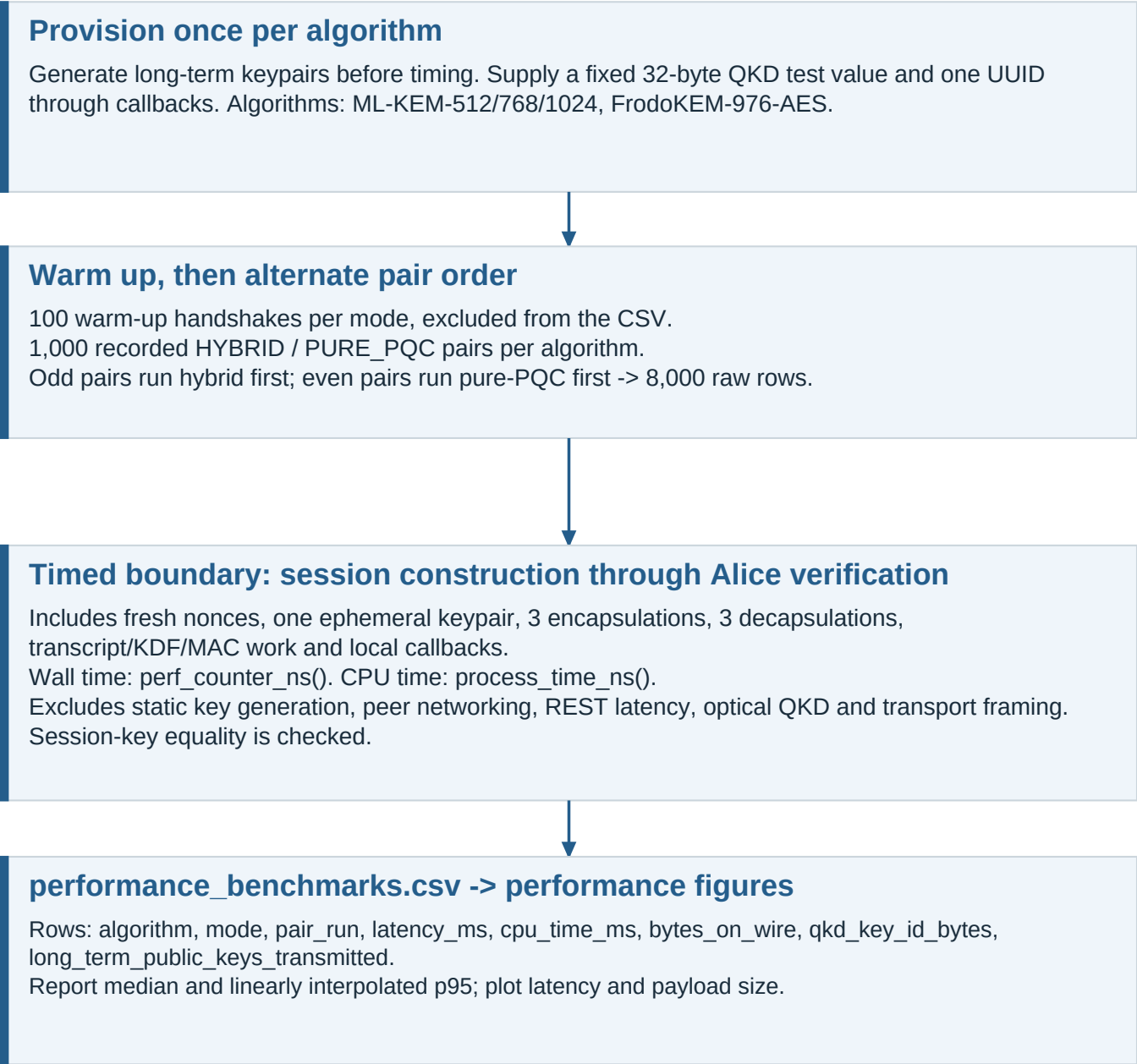


FIGURE 4. The policy layer is implemented, but control scope and atomicity determine what it can enforce. Security mode is transcript-bound after the decision; no separate mode-negotiation field appears in the message tuples.

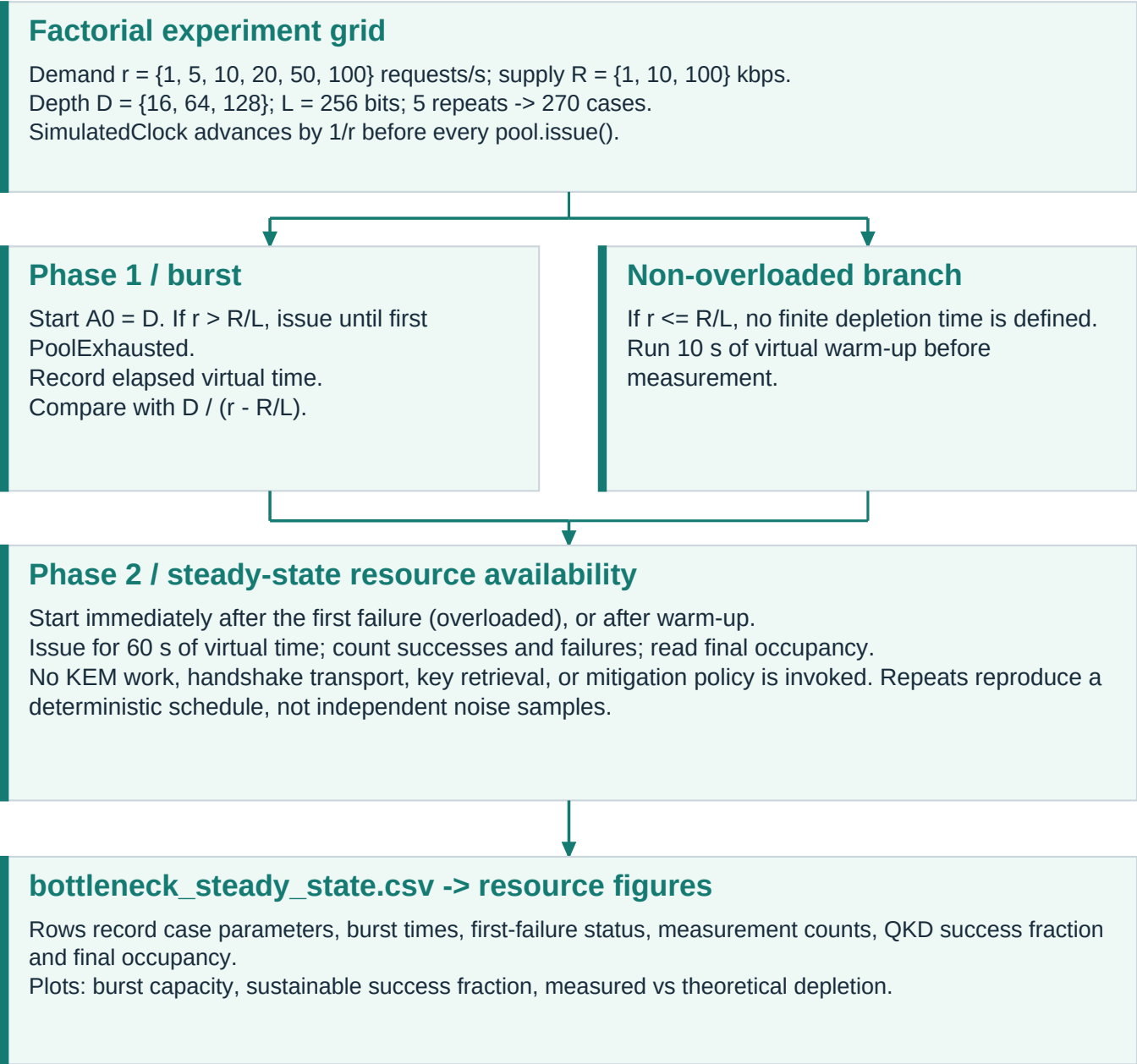
Experiment architecture and measurement boundaries

Cryptographic cost and QKD resource scarcity are measured independently; combining them into an end-to-end claim requires another experiment.

A / PAIRED CRYPTOGRAPHIC PERFORMANCE



B / TWO-PHASE QKD STARVATION SIMULATION



C / ANALYTICAL INTERPRETATION

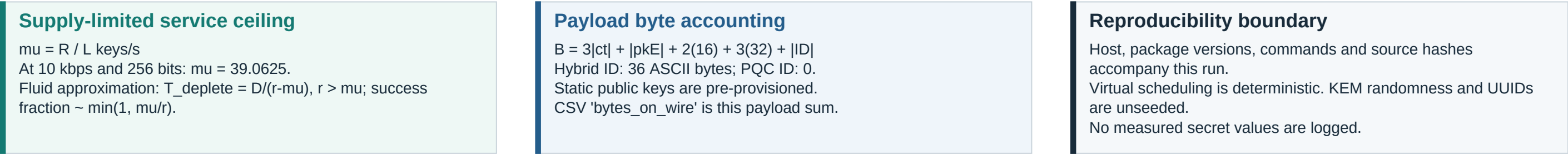


FIGURE 5. A fluid depletion formula is an analytical reference, not an exact identity for discrete arrivals and lazy refill. The experiment configuration JSON lists policies but the suite uses its own active parameter lists.

Module traceability, verification and research boundaries

A reviewable map from figure claims to source files, with the conditions needed to interpret the experimental evidence.

A / IMPLEMENTATION-TO-EVIDENCE MATRIX

MODULE / SOURCE	ARCHITECTURAL RESPONSIBILITY	EXISTING TEST EVIDENCE
protocol/hake.py	4-message role state; transcript hash; key confirmation	test_hake.py: 5 scenarios
crypto/kem.py	liboqs adapter; 4 selected KEM algorithms	test_kem.py: 4 algorithm round trips
crypto/kdf.py + combiner.py	ROKDF for HAKE; separate labelled HKDF prototype	test_combiner.py: 3 mode/derivation checks
qkd_mock/pool.py	Lazy refill; credits; pair-bound ID ledger; mutex	test_pool.py: 3 lifecycle/exhaustion checks
server.py + models.py	FastAPI endpoints, schemas and error mapping	test_api.py: 2 delivery/exhaustion checks
client.py + mitigations.py	HTTP callbacks; request quota; 5% admission floor	HAKE tests + 4 loopback REST checks [V]
benchmarks/suite.py	Paired local handshakes; two-phase pool simulation	Full execution and output checks [V]
runner.py + plot_*.py	Smoke timing; CSV-derived figures	Demo and both plotting commands [V]
store.py / cli.py / settings.py	JSON identity utility / dispatch / QKD_* environment	Store is separate from active harness wiring

B / SECURITY AND SYSTEM BOUNDARIES

Identity and local secrets

Trusted static public-key provisioning is assumed. KeyStore stores public/secret key hex in JSON and is not used by the benchmark. Secret state resides in Python/native memory; the implementation does not demonstrate key erasure or a hardware trust boundary.

Independent HKDF prototype

derive_keys(): hash(mode || 0x00 || transcript), then labelled extracts for static KEM, ephemeral KEM and QKD (or explicit-pqc-only). Labelled expands produce session, client-confirmation and server-confirmation keys. Only the smoke path uses this combiner.

Claims requiring separate proof

Code traces and successful tests do not establish CK01 security, exact paper conformance or information-theoretic secrecy. The repository security-analysis document is a research outline. Match protocol labels, adversary model and combiner assumptions to [R1] before importing its claims.

[V] VERIFIED EXECUTION

17 tests passed | 4 REST-backed HAKE checks | 8,000 benchmark rows | 270 sweep cases | 20 smoke rows | 5 result plots

C / REFERENCES AND REPRODUCTION

[R1] Clermont & Henrich (2026). The Best of Both Worlds: Hybrid Authenticated Key Exchange for QKD(N) without Signatures. Cryptology ePrint 2026/1231. <https://eprint.iacr.org/2026/1231> (research reference; exact implementation correspondence unverified).

[R2] ETSI GS QKD 014 V1.1.1 (2019-02), REST-based key delivery API. Official ETSI PDF linked in the artifact manifest.

Rebuild: `uv run docs/architecture/generate_architecture.py` | Execute: `.venv/bin/python docs/architecture/run_verification.py`

FIGURE 6. This atlas describes the inspected repository and the executed experiments. Diagram generation and verification are companion scripts; the protocol implementation and historical result files were not modified.